
Hands-On Machine Learning

Phong cảnh Machine Learning

Giáo án lý thuyết — Chương 1 (Aurélien Géron, 2nd ed.)

Biên soạn:

ĐẶNG TẤN QUÍ

SĐT: 0377 122 210

2026

Mục lục

Lời nói đầu	3
1 Chương 1: Phong cảnh Machine Learning	4
1.1 Machine Learning là gì?	4
1.2 Tại sao nên dùng Machine Learning?	5
1.3 Ví dụ về ứng dụng	7
1.4 Các loại hệ thống Machine Learning	8
1.4.1 Supervised và Unsupervised Learning	8
1.4.2 Batch Learning và Online Learning	11
1.4.3 Instance-Based và Model-Based Learning	12
2 Những thách thức chính của Machine Learning	17
2.1 Không đủ dữ liệu training	17
2.2 Hiệu quả phi lý của dữ liệu	17
2.3 Dữ liệu training không đại diện	18
2.4 Ví dụ về sampling bias	18
2.5 Dữ liệu kém chất lượng	18
2.6 Tính năng không liên quan	18
2.7 Overfitting (học vẹt)	19
2.8 Underfitting (học không đủ)	20
3 Kiểm tra và xác nhận	21
4 Bài tập (tóm tắt)	22

Lời nói đầu

Cách đọc tài liệu

Giáo án này tóm tắt **Chương 1** sách *Hands-On Machine Learning* (Géron, 2nd ed.), theo template `toan_dai_hoc/nhom_5_xac_suat_thong_ke`. Mỗi khối màu tương ứng một vai trò: **động cơ** (tại sao học), **định nghĩa**, **ý nghĩa**, **phương pháp**, **ví dụ**, **công thức**, **lưu ý**, **định lý**, **tổng kết**.

Nguồn: bản dịch PDF + hình từ sách gốc + [notebook Chương 1](#).

Đặng Tấn Quý

1 Chương 1: Phong cảnh Machine Learning

Động cơ — Chương này dạy gì?

Chương 1 là **bản đồ** trước khi code: bạn cần nắm thuật ngữ, phân loại hệ thống ML và quy trình dự án. Đây là chương *ít code nhất* trong sách — hãy đảm bảo hiểu rõ trước khi sang Chương 2.

Roadmap chương:

1. ML là gì, tại sao dùng, ứng dụng điển hình.
2. Phân loại hệ thống: supervised/unsupervised/RL; batch/online; instance-based/model-based.
3. Ví dụ end-to-end: GDP $\rightarrow$ hạnh phúc (Linear Regression + k-NN).
4. Thách thức: dữ liệu xấu, overfitting/underfitting.
5. Đánh giá: train/validation/test, hyperparameter, định lý “không bữa trưa miễn phí”.

ML đã có mặt trong đời sống

Nhiều người nghĩ ML = robot; thực tế ML đã **len lỏi** vào sản phẩm hàng ngày từ thập niên 1990: **bộ lọc thư rác**, gợi ý phim/nhạc, tìm kiếm giọng nói, nhận dạng ký tự (OCR)... Không phải tương lai xa — đã chạy âm thầm hàng trăm triệu lần mỗi ngày.

1.1 Machine Learning là gì?

Học máy (Machine Learning)

Định nghĩa ngắn: ML là khoa học (và nghệ thuật) **lập trình máy tính để học từ dữ liệu**, thay vì viết từng quy tắc bằng tay.

Arthur Samuel (1959): “Lĩnh vực nghiên cứu cho phép máy tính học mà *không cần lập trình rõ ràng*.”

Tom Mitchell (1997): Chương trình **học** kinh nghiệm E trên nhiệm vụ T , đo bằng P , nếu hiệu suất trên T **cải thiện** theo E .

Thuật ngữ cốt lõi (ví dụ spam filter)

- **Training set:** email spam + email ham đã gắn nhãn.
- **Training instance / sample:** một email.
- **Nhiệm vụ T :** phân loại email mới (spam/ham).
- **Kinh nghiệm E :** dữ liệu training.
- **Độ đo P :** accuracy — tỷ lệ phân loại đúng.

Không phải cứ có dữ liệu là ML

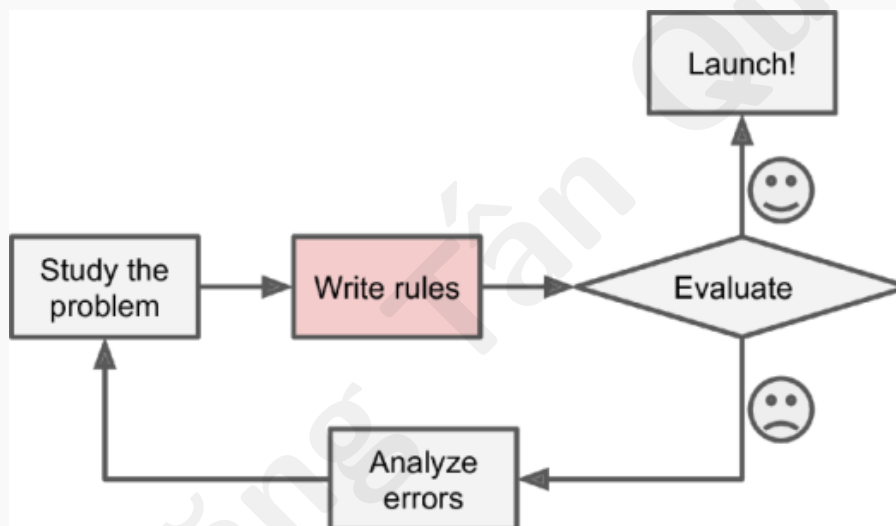
Tải bản sao Wikipedia về máy **không** làm máy “thông minh hơn” ở bất kỳ tác vụ cụ thể nào — chỉ là lưu trữ, không có quá trình **học** cải thiện P trên T .

1.2 Tại sao nên dùng Machine Learning?

Cách truyền thống: spam filter thủ công

Quy trình:

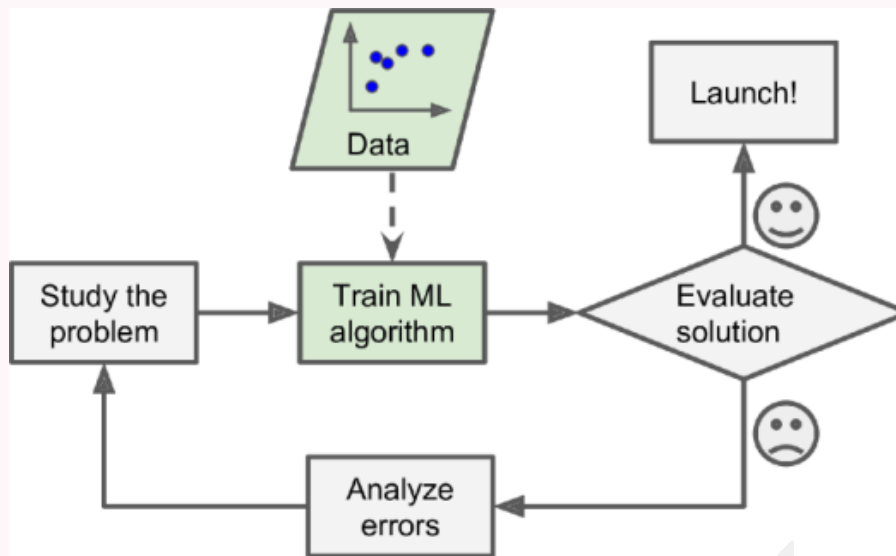
1. Quan sát mẫu spam, liệt kê từ/cụm hay gặp (“4U”, “miễn phí”...).
 2. Viết quy tắc: nếu gặp mẫu $\Rightarrow$ gắn cờ spam.
 3. Kiểm thử, bổ sung quy tắc mới mãi.
- $\Rightarrow$ Danh sách quy tắc dài, **khó bảo trì**, khó theo kịp spam đổi chiều.



Hình 1: Hình 1-1. Cách tiếp cận truyền thống

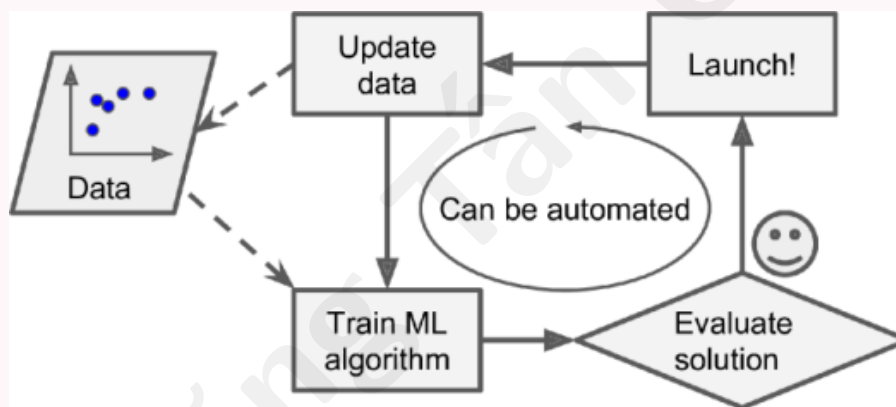
Cách ML: học từ dữ liệu

Bộ lọc ML **tự tìm** từ/cụm là predictor tốt (tần suất bất thường trong spam so với ham). Code ngắn hơn, dễ bảo trì, thường chính xác hơn.



Hình 2: Hình 1-2. Cách tiếp cận Machine Learning

Khi spammer đổi “4U” → “Dành cho U”: lập trình truyền thống phải **sửa tay**; ML **tự nhận** mẫu mới và gắn cờ.



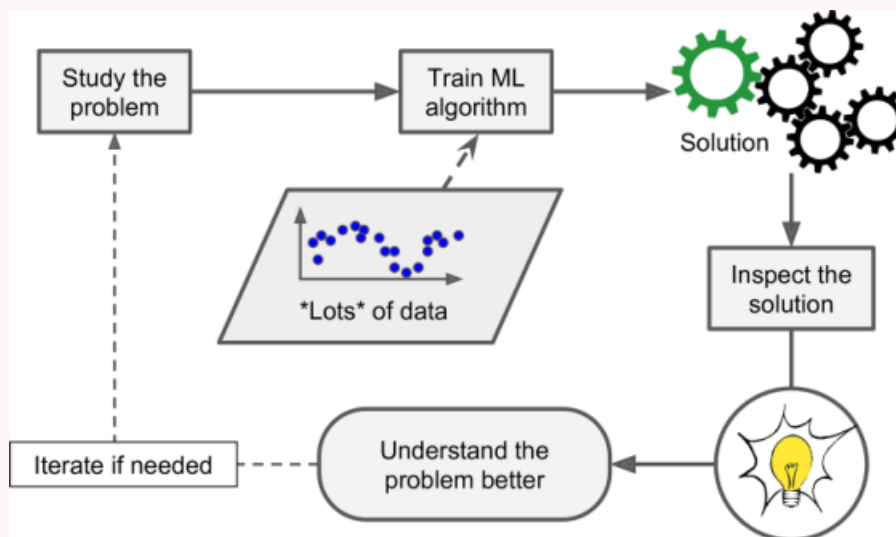
Hình 3: Hình 1-3. Tự động thích ứng với thay đổi

Bài toán quá phức tạp: nhận dạng giọng nói

Không thể viết quy tắc “âm cao = chữ hai” cho hàng nghìn từ × hàng triệu người nói × nhiều môi trường. Giải pháp: thuật toán **tự học** từ hàng loạt bản ghi mẫu.

ML giúp con người học

Có thể **kiểm tra** model đã học gì (ví dụ: danh sách từ spam quan trọng nhất) ⇒ hiểu bài toán sâu hơn, phát hiện mẫu bất ngờ — **khai thác dữ liệu**.



Hình 4: Hình 1-4. Machine Learning giúp con người học

Khi nào nên chọn ML?

- Quy tắc tay quá dài / cần tuning liên tục.
- Không có thuật toán cổ điển tốt (giọng nói, thị giác...).
- Môi trường **biến động**, cần thích ứng dữ liệu mới.
- Cần **insight** từ khối dữ liệu lớn.

1.3 Ví dụ về ứng dụng

Một số nhiệm vụ ML điển hình

- **Phân loại ảnh** sản phẩm lỗi (CNN, Ch.14).
- **Phân đoạn khối u** trên MRI (semantic segmentation).
- **Phân loại văn bản**, phát hiện comment xúc phạm (NLP, Ch.16).
- **Regression** dự báo doanh thu, giá nhà (Linear/Polynomial Regression, SVM, RF, NN).
- **Nhận dạng giọng nói** (RNN/CNN/Transformer, Ch.15–16).
- **Phát hiện gian lận** thẻ tín dụng (anomaly detection, Ch.9).
- **Clustering** phân khúc khách hàng (Ch.9).
- **Trực quan hóa / giảm chiều** dataset phức tạp (Ch.8).
- **Gợi ý sản phẩm** (collaborative filtering, NN).
- **Reinforcement Learning**: bot game, AlphaGo (Ch.18).

1.4 Các loại hệ thống Machine Learning

Ba trục phân loại

Có thể kết hợp tự do:

1. **Giám sát:** supervised / unsupervised / semi-supervised / reinforcement.
2. **Cách học:** batch (offline) vs online (incremental).
3. **Cách khái quát:** instance-based vs model-based.

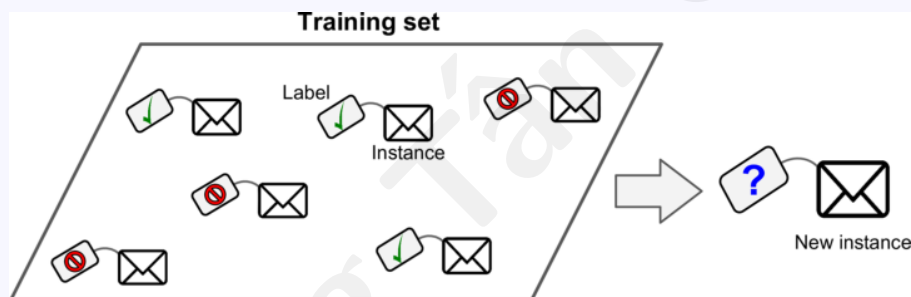
Ví dụ: spam filter hiện đại = online + model-based + supervised (deep net).

1.4.1 Supervised và Unsupervised Learning

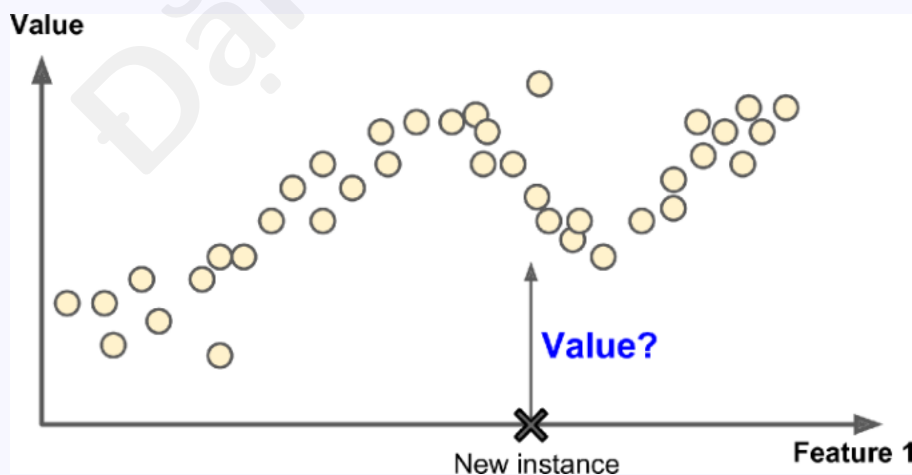
Supervised learning (học có giám sát)

Training set kèm **nhãn** (đáp án mong muốn).

- **Classification:** nhãn rời rạc (spam/ham, loài hoa...).
- **Regression:** nhãn số (giá nhà, GDP $\rightarrow$ hạnh phúc...).



Hình 5: Hình 1-5. Training set có nhãn (spam)

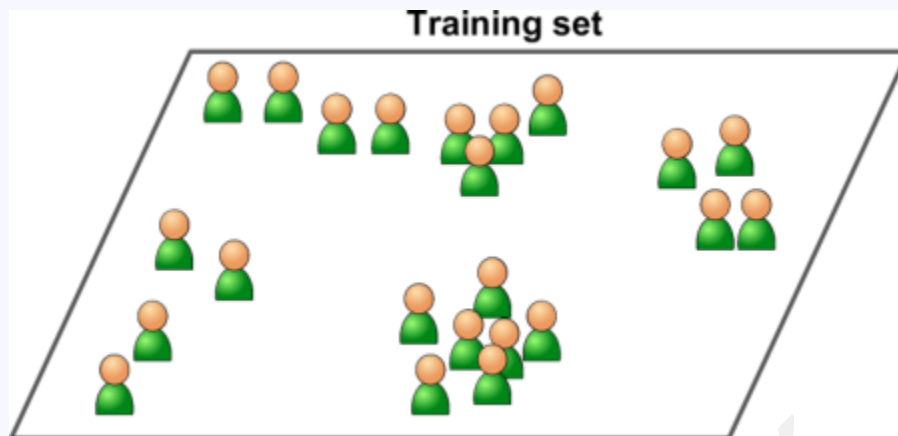


Hình 6: Hình 1-6. Bài toán regression

Thuật toán tiêu biểu: k-NN, Linear/Logistic Regression, SVM, Decision Tree, Random Forest, Neural Network.

Unsupervised learning (học không giám sát)

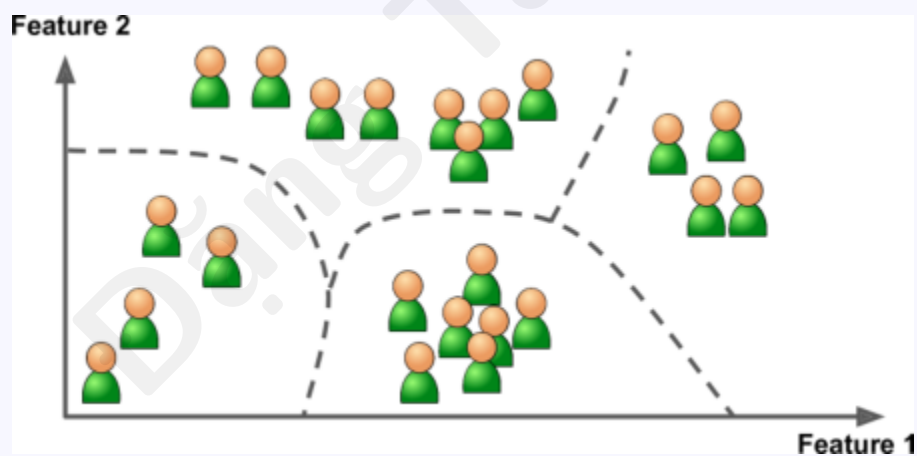
Dữ liệu **không nhãn**; hệ thống tự tìm cấu trúc.



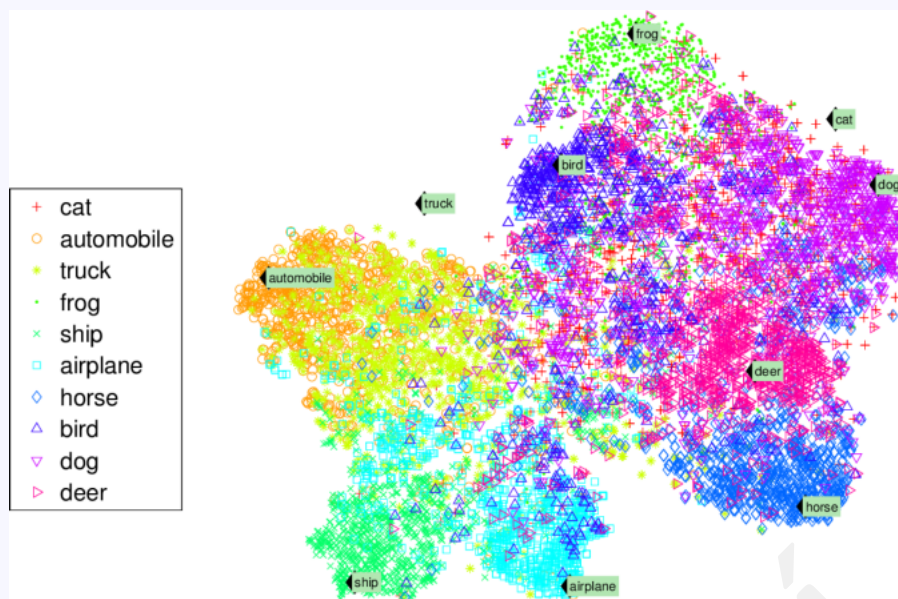
Hình 7: Hình 1-7. Training set không nhãn

Nhiệm vụ chính:

- **Clustering** (K-Means, DBSCAN, HCA) — nhóm khách blog...
- **Anomaly / novelty detection** — gian lận, lỗi sản xuất.
- **Visualization & dimensionality reduction** (PCA, t-SNE...).
- **Association rule learning** (Apriori, Eclat).



Hình 8: Hình 1-8. Clustering



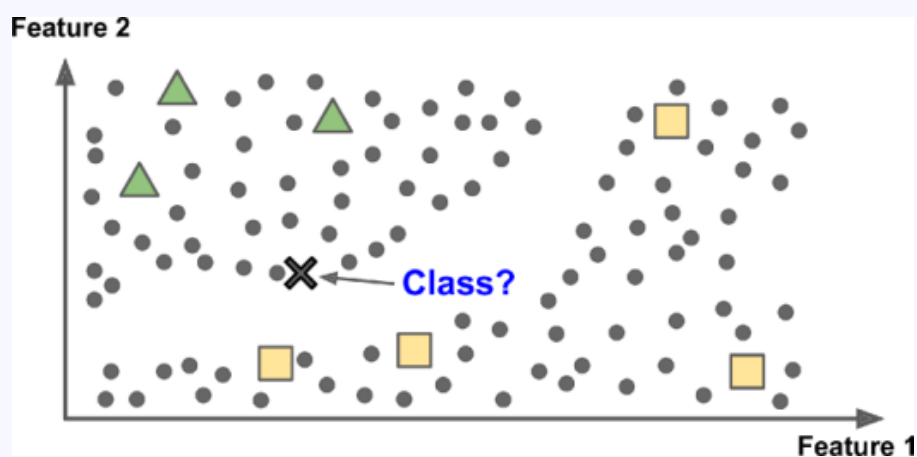
Hình 9: Hình 1-9. Trực quan hóa t-SNE



Hình 10: Hình 1-10. Phát hiện bất thường

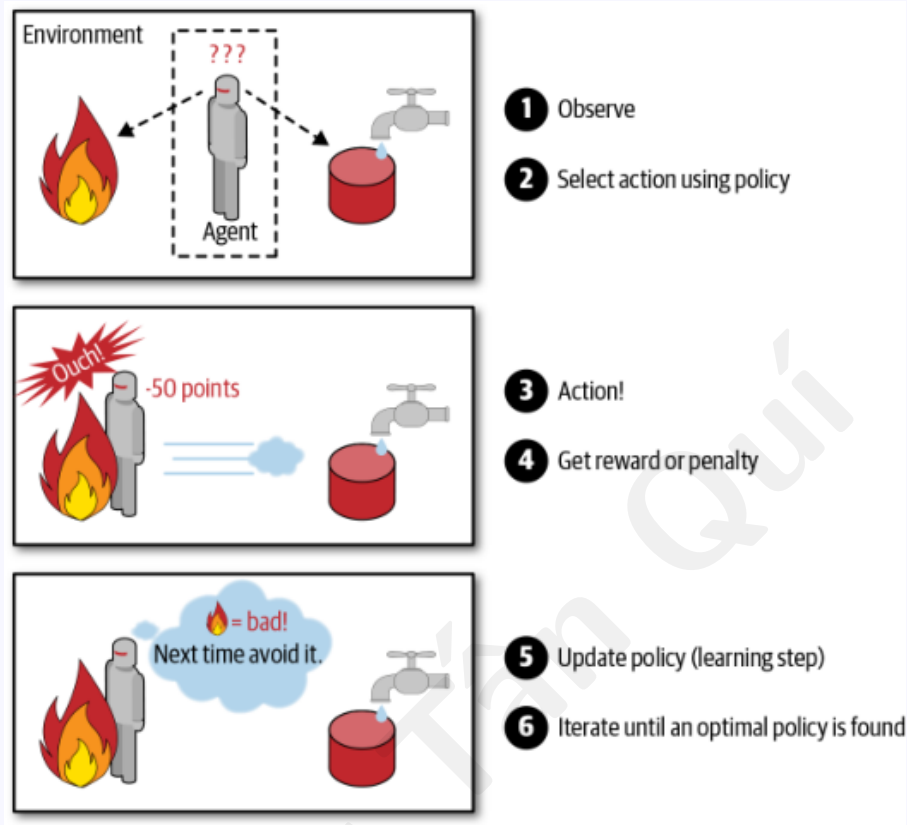
Semi-supervised và Reinforcement Learning

Semi-supervised: nhiều dữ liệu **không nhãn**, ít dữ liệu có nhãn — nhãn thưa giúp phân loại điểm mới chính xác hơn.



Hình 11: Hình 1-11. Học bán giám sát

Reinforcement Learning (RL): agent quan sát môi trường, chọn hành động, nhận reward/penalty, học policy tối đa hóa phần thưởng lâu dài (robot đi bộ, AlphaGo).



Hình 12: Hình 1-12. Reinforcement Learning

1.4.2 Batch Learning và Online Learning

Batch learning (học theo lô)

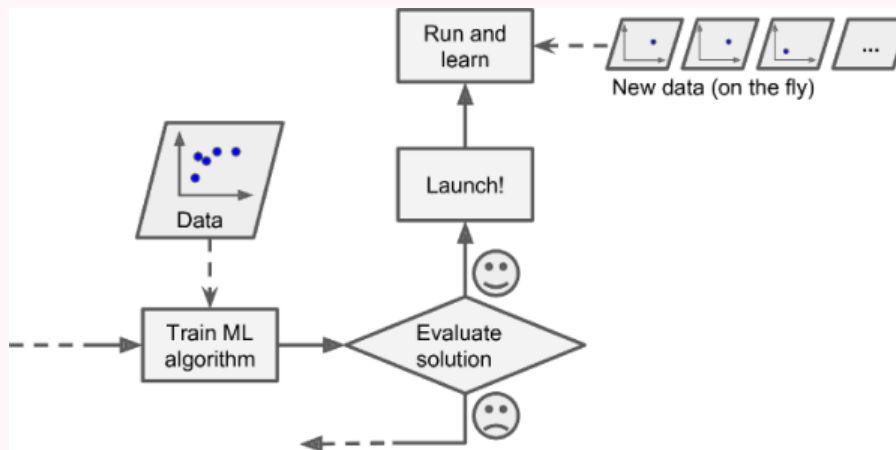
Training **toàn bộ** dữ liệu một lần (thường offline), rồi triển khai model **cố định**. Dữ liệu mới $\Rightarrow$ train lại từ đầu trên dataset đầy đủ.

Ưu: đơn giản, dễ tự động hóa retrain hàng ngày/tuần.

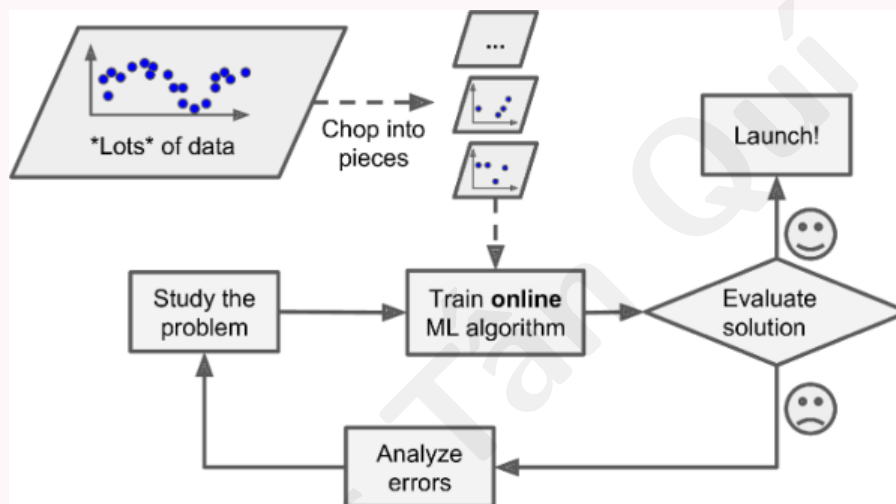
Nhược: chậm, tốn tài nguyên; không phù hợp dữ liệu thay đổi rất nhanh (giá cổ phiếu) hoặc thiết bị yếu (điện thoại, rover sao Hỏa).

Online learning (học trực tuyến / tăng dần)

Cập nhật model **tuần tự** từng instance hoặc mini-batch; mỗi bước nhanh, rẻ.



Hình 13: Hình 1-13. Online learning trong production



Hình 14: Hình 1-14. Out-of-core learning (dataset khổng lồ)

Phù hợp: **luồng dữ liệu** liên tục, tài nguyên hạn chế. Có thể xử lý dataset **không vừa RAM** (load từng phần).

Learning rate: cao $\Rightarrow$ thích ứng nhanh nhưng quên cũ; thấp $\Rightarrow$ ổn định nhưng chậm.

Rủi ro online learning

Dữ liệu xấu (cảm biến hỏng, spam SEO) có thể **làm hỏng** model đang chạy production. Cần **giám sát** hiệu suất, tắt học khi suy giảm, có thể rollback.

1.4.3 Instance-Based và Model-Based Learning

Instance-based learning

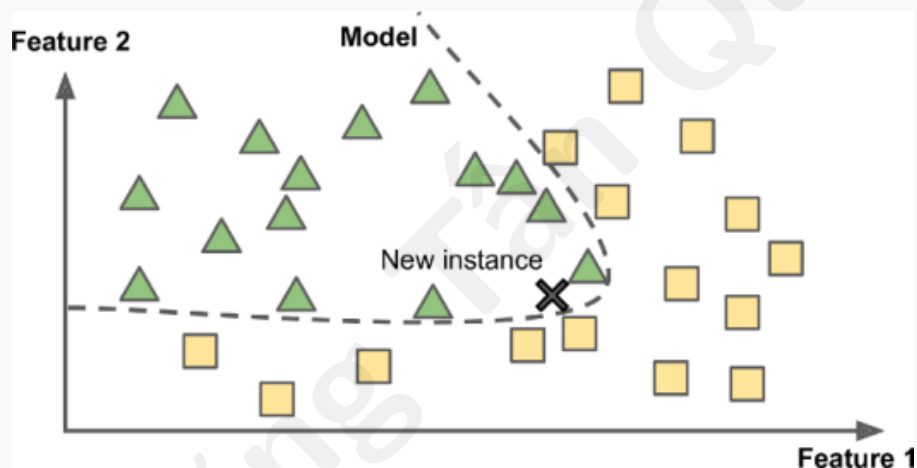
Học thuộc (một phần) training set; dự đoán điểm mới bằng **độ tương tự** với ví dụ đã thấy (ví dụ: đếm từ chung, k-NN — lớp của k láng giềng gần nhất).



Hình 15: Hình 1-15. Instance-based learning (k-NN phân loại tam giác)

Model-based learning

Xây **model** (công thức có tham số) từ dữ liệu, rồi dùng model dự đoán điểm mới.



Hình 16: Hình 1-16. Model-based learning

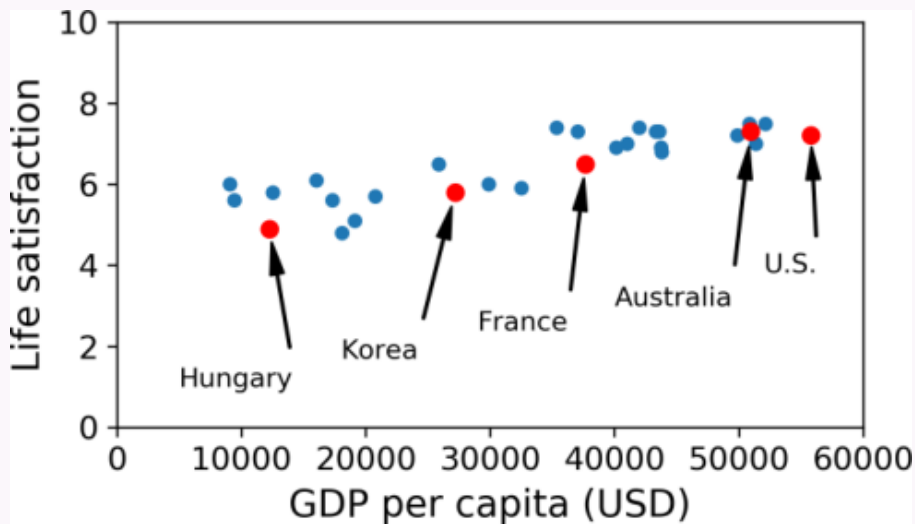
Ví dụ 1-1: GDP vs mức hài lòng cuộc sống

Câu hỏi: Tiền có làm người ta hạnh phúc hơn?

Bước 1 — Thu thập: OECD (Better Life Index) + IMF (GDP bình quân đầu người).

Bảng 1: Bảng 1-1. GDP và hài lòng (trích)

Quốc gia	GDP (USD)	Hài lòng
Hungary	12 240	4,9
Hàn Quốc	27 195	5,8
Pháp	37 675	6,5
Úc	50 962	7,3
Hoa Kỳ	55 805	7,2



Hình 17: Hình 1-17. GDP vs life satisfaction — có xu hướng tuyến tính

Bước 2 — Chọn model: hàm tuyến tính 1 biến (GDP).

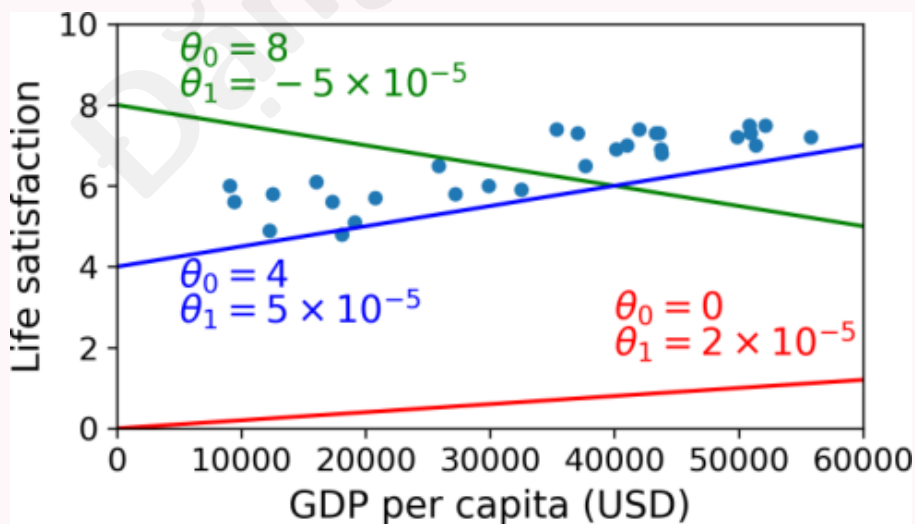
Phương trình 1-1 — Model tuyến tính

$$\text{life_satisfaction} = \theta_0 + \theta_1 \times \text{GDP_per_capita} \quad (1)$$

θ_0, θ_1 : **tham số model** (parameters). Chọn model $\neq$ train model $\neq$ model đã train sẵn sàng dự đoán.

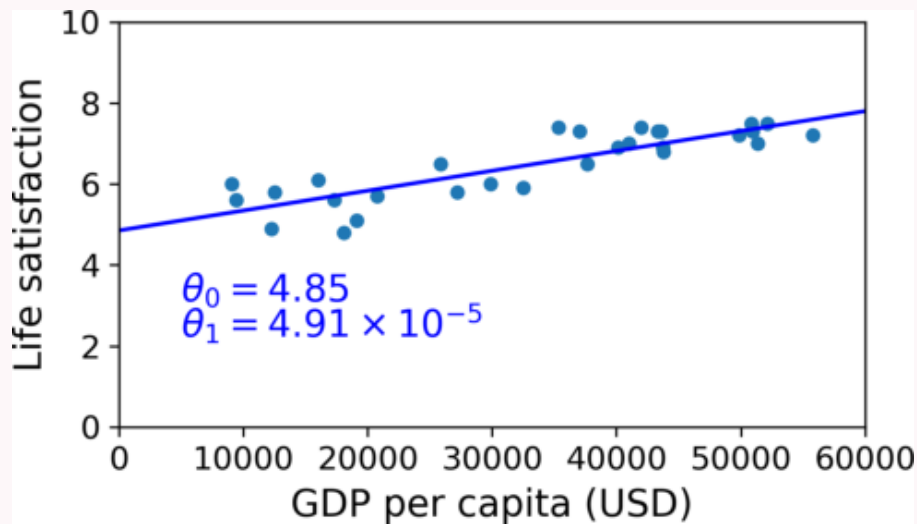
Training = tìm θ tối ưu

Do **hàm chi phí** (khoảng cách dự đoán vs thực tế). **Linear Regression** tìm θ cực tiểu cost.



Hình 18: Hình 1-18. Các đường thẳng tuyến tính khác nhau

Kết quả: $\theta_0 = 4,85$, $\theta_1 = 4,91 \times 10^{-5}$.



Hình 19: Hình 1-18. Model sau training

Dự đoán Cyprus (GDP $\approx$ 22 587 USD):

$$\hat{y} = 4,85 + 22\,587 \times 4,91 \times 10^{-5} \approx 5,96$$

Code Ví dụ 1-1 (handson-ml2)

Notebook: https://github.com/ageron/handson-ml2/blob/master/01_the_machine_learning_landscape.ipynb

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import sklearn.linear_model

# ... prepare_country_stats(), tại CSV tu GitHub ...

country_stats = prepare_country_stats(oecd_bli, gdp_per_capita)
X = np.c_[country_stats["GDP_per_capita"]]
y = np.c_[country_stats["Life_satisfaction"]]

country_stats.plot(kind='scatter', x="GDP_per_capita", y='Life_satisfaction')
plt.show()

model = sklearn.linear_model.LinearRegression()
model.fit(X, y)

X_new = [[22587]] # Cyprus
print(model.predict(X_new)) # ~5.96
```

k-NN (chỉ đổi model): lấy trung bình hạnh phúc của 3 nước có GDP gần Cyprus nhất $\Rightarrow \approx 5,77$.

```
import sklearn.neighbors
model = sklearn.neighbors.KNeighborsRegressor(n_neighbors=3)
model.fit(X, y)
```

```
print(model.predict(X_new)) # ~5.77
```

Quy trình ML điển hình (tóm tắt)

1. **Nghiên cứu** dữ liệu.
2. **Chọn model** phù hợp.
3. **Train** trên training set (tối ưu tham số).
4. **Suy luận** (inference) trên dữ liệu mới — hy vọng khái quát tốt.

Chi tiết end-to-end ở **Chương 2**.

Đặng Tấn Quý

2 Những thách thức chính của Machine Learning

Hai nguồn lỗi

Nhiệm vụ = chọn thuật toán + train trên dữ liệu. Lỗi đến từ **thuật toán xấu** hoặc **dữ liệu xấu** — thực tế thường là dữ liệu.

2.1 Không đủ dữ liệu training

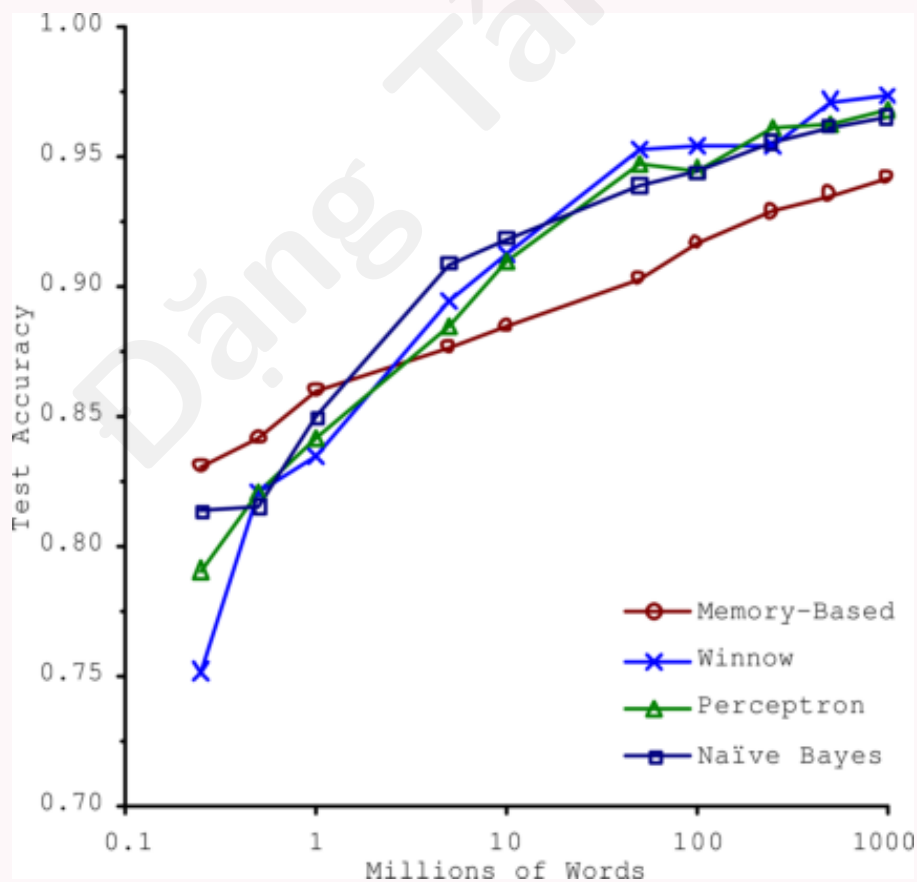
Trẻ học 1 quả táo vs máy học

Trẻ chỉ cần vài lần chỉ “táo”; ML cần **hàng nghìn** mẫu (bài đơn giản) đến **hàng triệu** (ảnh, giọng nói), trừ khi transfer learning từ model có sẵn.

2.2 Hiệu quả phi lý của dữ liệu

Banko & Brill (2001) — “More data beats clever algorithms”

Với đủ dữ liệu, nhiều thuật toán (kể cả đơn giản) cho kết quả **gần như nhau** trên phân loại ngôn ngữ. Nên cân nhắc đầu tư **thu thập dữ liệu** vs tinh chỉnh thuật toán.

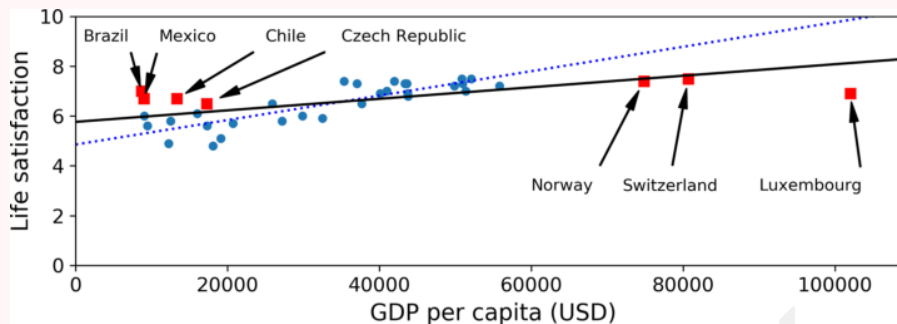


Hình 20: Hình 1-20. Hiệu quả phi lý của dữ liệu

2.3 Dữ liệu training không đại diện

Training set phải giống production

Muốn khái quát tốt, training phải **đại diện** cho tình huống thực tế. Thiếu quốc gia trong bộ GDP $\Rightarrow$ model tuyến tính **sai lệch** (nước rất giàu/rất nghèo).



Hình 21: Hình 1-21. Training set không đại diện

Sampling bias: mẫu lệch do cách thu thập (không chỉ do mẫu nhỏ).

2.4 Ví dụ về sampling bias

Literary Digest 1936 & YouTube funk

1936: Thăm dò 10 triệu người qua danh bạ điện thoại/tạp chí (giàu, Cộng hòa) $\Rightarrow$ dự đoán Landon thắng; thực tế Roosevelt thắng (chỉ ~25% phản hồi).

YouTube funk: Kết quả tìm kiếm thiên nghệ sĩ nổi tiếng / “funk carioca” (Brazil) $\neq$ toàn bộ funk James Brown.

2.5 Dữ liệu kém chất lượng

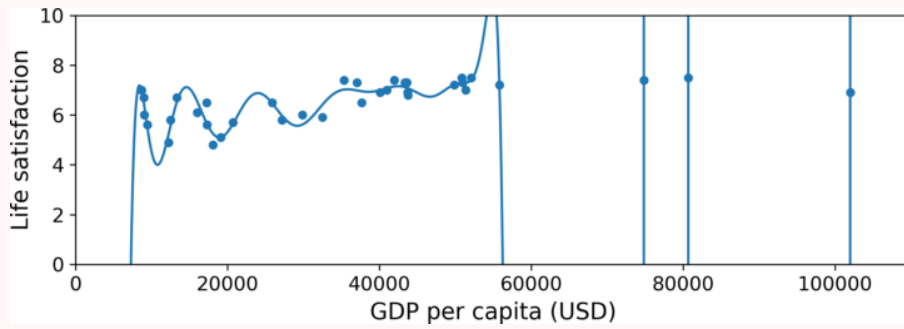
Garbage in, garbage out

Lỗi đo, nhiễu, outlier $\Rightarrow$ model học sai mẫu. Data scientist dành **nhiều thời gian** làm sạch: xóa/sửa outlier, xử lý missing values, loại bỏ duplicate gần trùng.

2.6 Tính năng không liên quan

Feature engineering

Model chỉ học được nếu có **feature đúng**, không quá nhiều feature vô nghĩa (ví dụ: tên quốc gia có chữ “w” vs hạnh phúc).



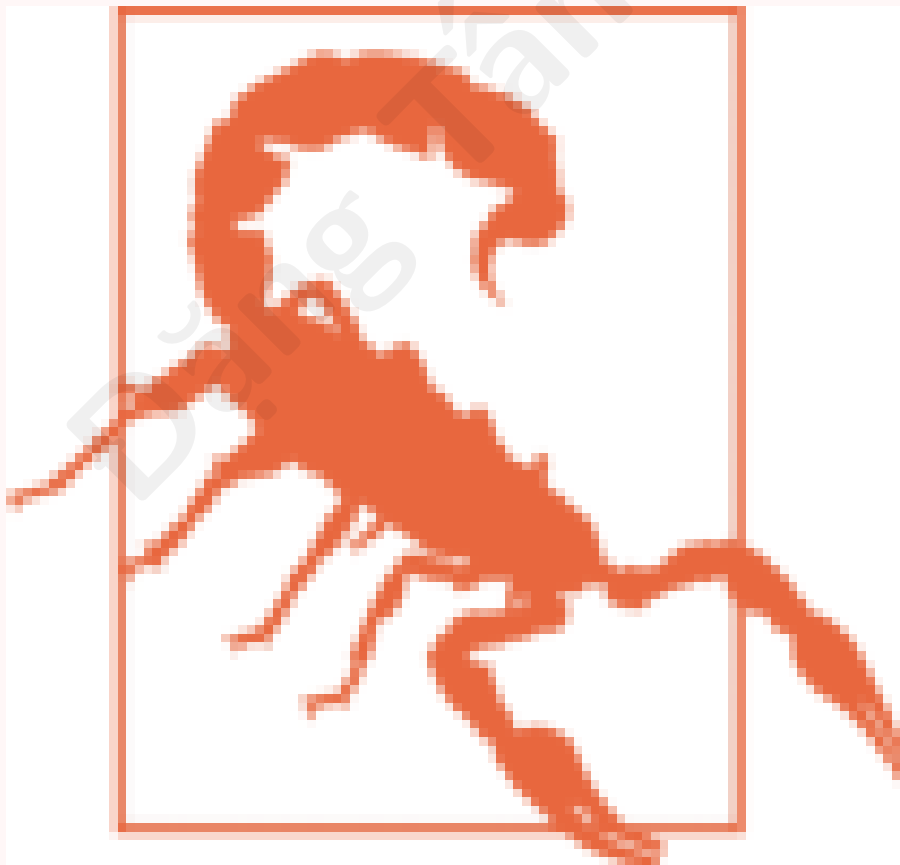
Hình 22: Hình 1-22. Feature không liên quan (minh họa)

Ba hướng: (1) **feature selection**; (2) **feature extraction** (PCA...); (3) thu thập feature mới.

2.7 Overfitting (học vẹt)

Overfitting = tốt trên train, tệ trên mới

Model **quá phức tạp** so với lượng/chất lượng dữ liệu $\Rightarrow$ học cả **những** (ví dụ: quy tắc ngẫu nhiên “quốc gia có w”).

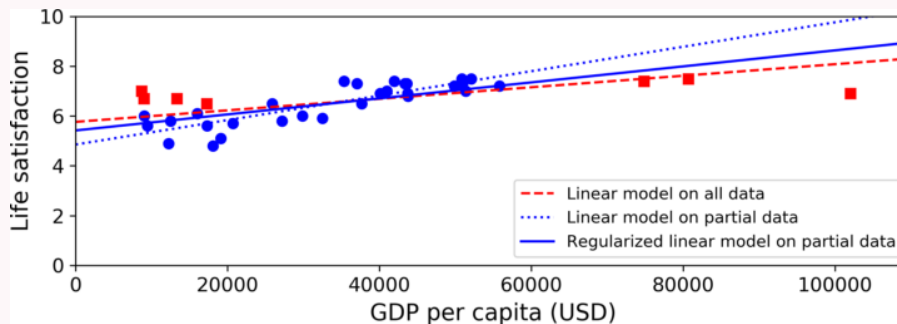


Hình 23: Hình 1-23. Overfitting trên GDP–hạnh phúc

Khắc phục: thu gọn model; thêm dữ liệu; giảm nhiễu; **regularization**.

Regularization — ép model đơn giản hơn

Hạn chế độ lớn của $\theta_1 \Rightarrow$ đường gần ngang hơn, **không khớp hoàn hảo** train nhưng **khái quát** tốt hơn trên điểm mới.



Hình 24: Hình 1-24. Regularization

Hyperparameter: tham số của *thuật toán học* (không học từ data), ví dụ mức regularization — đặt **trước** training.

2.8 Underfitting (học không đủ)

Underfitting

Model **quá đơn giản** (ví dụ: đường thẳng cho quan hệ phi tuyến) $\Rightarrow$ sai cả trên train lẫn mới.

Khắc phục: model phức tạp hơn; feature tốt hơn; giảm regularization.

Bước lùi — Tóm tắt nhanh

- ML: học từ dữ liệu, không viết quy tắc tay.
- Hệ thống tốt cần **dữ liệu đủ, sạch, đại diện + feature phù hợp + model vừa đủ** (không over/underfit).
- Phân loại: supervised/unsupervised/RL; batch/online; instance/model-based.

3 Kiểm tra và xác nhận

Đánh giá và chọn model

Training error thấp nhưng **generalization error cao** $\Rightarrow$ overfitting.

Tuning hyperparameter (ví dụ regularization): thử nhiều giá trị — **không** được chọn model tốt nhất rồi báo cáo lỗi trên cùng tập đó (sẽ lạc quan).

Hold-out validation: tách **validation set** khỏi training $\Rightarrow$ chọn model/hyperparameter $\Rightarrow$ train lại trên full train+val $\Rightarrow$ đánh giá cuối trên **test set** (chỉ dùng **một lần**).



Hình 25: Hình 1-25. Train / validation / test

Cross-validation: nhiều fold nhỏ khi validation set quá nhỏ hoặc train bị cắt nhiều.

Dữ liệu production khác training

Ví dụ: train trên ảnh hoa web, deploy trên ảnh chụp điện thoại — **val/test phải đại diện production**. Có thể fine-tune một phần cuối trên ảnh thật.

Định lý “Không bữa trưa miễn phí” (NFL, Wolpert 1996)

Không có giả định nào về dữ liệu thì **không có lý do** ưu tiên model A hơn B. Mỗi dataset có model phù hợp riêng (tuyến tính vs neural net...). Thực tế: đưa ra **giả định hợp lý** và so sánh một **tập model ứng viên** hạn chế.

4 Bài tập (tóm tắt)

Câu hỏi ôn tập cuối chương

1. Định nghĩa ML theo Mitchell (T, E, P)?
2. Hai loại supervised task? Ví dụ?
3. Clustering thuộc loại học nào?
4. Batch vs online learning?
5. Instance-based vs model-based?
6. Overfitting là gì? Ba cách xử lý?
7. Validation set dùng để làm gì?
8. Định lý NFL nói gì?

Tổng kết Chương 1

- **ML** = học từ dữ liệu; spam filter là ví dụ kinh điển.
- **Phân loại hệ thống**: supervised / unsupervised / semi / RL; batch / online; instance / model-based.
- **Pipeline**: thu thập → chọn model → train → đánh giá → triển khai.
- **Ví dụ 1-1**: Linear Regression GDP-hạnh phúc; so sánh k-NN.
- **Rủi ro**: thiếu/không đại diện/nhiều dữ liệu; overfitting/underfitting; cần train/-val/test đúng cách.

Sẵn sàng cho **Chương 2** — dự án ML end-to-end bằng code.